

Approximation de la fonction de valeur et réseaux DQN

Marin Ferecatu

Notes de cours

UE RCP211, Séance RL No. 3

CNAM Paris

Document rédigé en L^AT_EX

10 avril 2026

Table des matières

Avant-propos	2
Notations globales	3
1 Introduction	4
1.1 Introduction	4
1.2 Spécificités de la VFA	5
2 Approximation de la fonction de valeur	6
2.1 Principe de la méthode	6
2.2 Apprentissage avec un oracle	6
2.3 Rappel : descente de gradient stochastique	6
2.4 Approximation linéaire de la fonction de valeur	8
2.5 Monte Carlo avec approximation de la fonction de valeur	9
2.6 TD Learning avec approximation de la fonction de valeur	11
2.7 Contrôle avec approximation de la fonction de valeur	13
3 Deep Reinforcement Learning	17
3.1 Principe de la méthode	17
3.2 End to end learning of Q	17
3.3 Extensions	23
3.3.1 Double DQN	23
3.3.2 Prioritized Experience Replay	24
3.3.3 Dueling DQN	25
Conclusion	30
Bibliographie	30

Avant-propos

Ces notes poursuivent directement les deux polycopiés précédents : *Apprentissage par renforcement : cadre général et programmation dynamique* [1] et *Apprentissage par renforcement : prédiction et contrôle Model Free* [2]. On y suppose donc connus le cadre markovien, les notions de retour, de politique et de fonction de valeur, les équations de Bellman [1, section 2.10, p. 15], ainsi que les principales méthodes model-free tabulaires : Monte Carlo, TD(0), SARSA et Q-learning [2, chapitres 2 et 3].

Le deuxième polycopié se terminait par une transition vers *l'approximation de fonction et le deep RL* [2, section 3.3.9, p. 31]. C'est précisément ce point que l'on développe ici. L'idée générale est simple : dans un grand nombre de problèmes réalistes, il n'est plus possible de stocker une table $V(s)$ ou $Q(s, a)$ pour tous les états ou pour tous les couples état-action. Il faut donc remplacer la table par une fonction paramétrée capable à la fois de représenter des valeurs et de généraliser entre des situations voisines.

Le fil conducteur restera le même que dans les deux premiers textes. Les équations de Bellman ne disparaissent pas ; elles restent la boussole théorique. Ce qui change, c'est la manière de représenter les quantités apprises et la manière de les ajuster à partir de l'expérience.

Notations globales

Les notations des deux premiers polycopiés sont conservées. On ajoute ici quelques notations spécifiques à l'approximation de fonction et au deep RL.

$\hat{v}(s, \mathbf{w})$	approximation paramétrée de $v_\pi(s)$;
$\hat{q}(s, a, \mathbf{w})$	approximation paramétrée de $q_\pi(s, a)$ ou $q_*(s, a)$;
$\mathbf{w} \in \mathbb{R}^d$	vecteur de paramètres d'un modèle, souvent linéaire ;
$x(s) \in \mathbb{R}^d$	vecteur de caractéristiques d'un état s (<i>feature vector</i>) ;
$x(s, a) \in \mathbb{R}^d$	vecteur de caractéristiques du couple (s, a) ;
$\boldsymbol{\theta}$	paramètres d'un réseau de neurones ;
y_t	cible d'apprentissage (<i>target</i>) au temps t ;
$\mathcal{D}$	mémoire de rejeu, ou <i>replay buffer</i> ;
$\boldsymbol{\theta}^-$	paramètres du réseau cible (<i>target network</i>) ;
$\hat{A}(s, a)$	estimation d'un avantage (<i>advantage</i>) ;
δ_t	erreur TD produite par la cible utilisée.

Comme dans les deux premiers documents, le temps est discret. Les notations v_π , q_π , v_* et q_* désignent les fonctions théoriques, tandis que les symboles avec un chapeau désignent des approximations apprises.

Chapitre 1

Introduction

1.1 Introduction

Dans RL2, on a appris à estimer des fonctions de valeur à partir de trajectoires observées, sans connaître le modèle de l'environnement [2, section 1.2 et chapitre 2]. Les méthodes présentées étaient toutefois *tabulaires*. Autrement dit, on supposait pouvoir mémoriser explicitement une valeur par état, ou une valeur par couple (s, a) . Cette hypothèse est raisonnable pour des exemples élémentaires : petits mondes quadrillés, labyrinthes simples, jeux très réduits. Elle devient rapidement irréaliste lorsque l'espace d'états est grand, continu, ou décrit par des observations riches. Si l'état d'un agent est, par exemple, une image, un vecteur de mesures physiques ou la configuration continue d'un robot, l'idée même d'une table exhaustive perd tout réalisme.

C'est ici qu'apparaît le problème de **l'approximation de la fonction de valeur** (*value function approximation*, ou VFA). Au lieu d'apprendre séparément une valeur pour chaque état, on choisit une famille de fonctions dépendant d'un ensemble de paramètres $\mathbf{w}$, puis on ajuste ces paramètres pour obtenir une bonne approximation de la vraie fonction de valeur. On écrit typiquement

$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s) \quad \text{ou} \quad \hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a), \quad q_{*}(s, a).$$

Ce changement de représentation a une conséquence fondamentale. Dans les méthodes tabulaires étudiées dans le cours précédent, une transition observée depuis l'état $S_t = s$ conduisait à corriger essentiellement la case $V(s)$ ou $Q(s, a)$ correspondante. L'effet de la mise à jour restait donc local : apprendre quelque chose sur un état modifiait directement la valeur stockée pour cet état, mais n'affectait pas les autres états.

Avec l'approximation de fonction, la situation change complètement. On ne stocke plus une valeur indépendante pour chaque état ; on introduit une fonction paramétrée, par exemple $\hat{v}(s, \mathbf{w})$, où $\mathbf{w}$ désigne un vecteur de paramètres commun à tous les états. Lorsqu'une transition observée entraîne une mise à jour de $\mathbf{w}$, ce ne sont pas seulement les prédictions en s_t qui changent. Comme les mêmes paramètres interviennent dans le calcul de $\hat{v}(s, \mathbf{w})$ pour de nombreux états s , la correction appliquée à partir d'une seule observation modifie en général les valeurs prédites dans tout un ensemble d'états, parfois même dans l'ensemble de l'espace d'états.

C'est précisément cette propriété que l'on appelle *généralisation* (*generalization*). Elle est indispensable dès que l'espace d'états devient trop grand pour être traité de manière tabulaire : une expérience acquise dans certains états peut alors améliorer aussi les prédictions dans d'autres états non encore visités, à condition que la représentation choisie capture correctement les similarités pertinentes entre états. Mais cette mutualisation des paramètres introduit aussi une difficulté nouvelle. Comme les prédictions des différents états sont couplées par les mêmes paramètres $\mathbf{w}$, une mise à jour utile pour certains états peut détériorer les prédictions sur d'autres. Les corrections peuvent ainsi interférer entre elles, provoquer des oscillations, ralentir l'apprentissage, voire conduire à une divergence lorsque les cibles elles-mêmes

évoluent au cours de l'apprentissage. C'est en ce sens que l'approximation de fonction rend le problème potentiellement plus instable que dans le cas tabulaire.

On peut résumer l'idée ainsi : dans le cas tabulaire, apprendre est essentiellement local ; avec l'approximation de fonction, apprendre devient global. Le passage à l'approximation de fonction ne répond pas seulement à un problème de mémoire ; il transforme aussi la manière d'apprendre, car l'information extraite d'une observation peut désormais être réutilisée dans de nombreux états grâce à une représentation partagée.

La transition depuis le model-free tabulaire vers l'approximation de fonction prolonge naturellement ce qui était annoncé à la fin de RL2 [2, section 3.3.9]. Le pas suivant mènera ensuite au **Deep Q-Network** (DQN), où la fonction approximante n'est plus linéaire, mais un réseau de neurones profond.

1.2 Spécificités de la VFA

L'approximation de fonction possède plusieurs spécificités qui méritent d'être isolées.

D'abord, on ne cherche plus à stocker ni à apprendre explicitement, pour chaque état, une dynamique, une récompense, une valeur d'état, une valeur d'action-état ou une politique. Dans une approche tabulaire, chacune de ces quantités est en quelque sorte attachée à des états ou à des couples état-action particuliers. Avec l'approximation de fonction, on remplace ce stockage exhaustif par un nombre plus réduit de paramètres partagés. Le problème devient donc moins un problème de *mémorisation état par état* qu'un problème de *représentation*.

Cette idée conduit naturellement à la recherche d'une **représentation compacte**. L'objectif n'est pas seulement de réduire la taille de ce qui est stocké, mais surtout d'organiser l'information de façon à faire apparaître les ressemblances pertinentes entre situations. Deux états jugés proches du point de vue de la décision devraient produire des prédictions proches. Mieux généraliser signifie précisément ceci : ne pas apprendre séparément une multitude de détails propres à chaque état, mais mutualiser, dans une représentation commune, ce qui permet de prédire correctement à partir de l'expérience disponible.

Cette mutualisation a plusieurs conséquences importantes. La première est une réduction potentielle du coût mémoire : on ne conserve plus une quantité distincte pour chaque état ou chaque couple état-action, mais un ensemble de paramètres souvent beaucoup plus petit. La deuxième est une réduction possible du coût de calcul, car il n'est plus nécessaire de mettre à jour une très grande table explicitement indexée par les états. Enfin, on peut aussi espérer une meilleure efficacité en données : une observation recueillie dans un état donné ne sert plus seulement à corriger cet état, mais peut améliorer la représentation commune et donc les prédictions dans d'autres états semblables, y compris parfois dans des états encore peu ou pas visités. Il faut toutefois comprendre que cet avantage dépend entièrement de la qualité de la représentation choisie. Si celle-ci ne capture pas les bonnes similarités entre états, la généralisation sera mauvaise : des états qui devraient être distingués seront confondus, ou au contraire des états similaires ne partageront pas correctement l'information utile.

Une autre spécificité importante est que le RL avec approximation de fonction ressemble en partie à un problème de régression. Si l'on disposait d'un oracle donnant la vraie valeur de chaque état ou de chaque couple état-action, on pourrait ajuster les paramètres par une méthode proche de l'apprentissage supervisé. En pratique, cet oracle n'existe pas ; il faut fabriquer des cibles d'apprentissage à partir de l'expérience, par exemple à l'aide de retours Monte Carlo ou de cibles TD.

Enfin, l'approximation de fonction devient particulièrement délicate lorsque trois ingrédients sont réunis : approximation, bootstrap et apprentissage off-policy. Cette combinaison est souvent appelée *deadly triad* en anglais. On n'en fera pas ici une étude approfondie, mais il faut déjà retenir qu'elle joue un rôle central dans les difficultés de Q-learning avec approximation et dans la motivation de DQN.

Chapitre 2

Approximation de la fonction de valeur

2.1 Principe de la méthode

Le principe général consiste à choisir une famille de fonctions paramétrées, puis à chercher des paramètres qui rendent cette fonction proche de la vraie fonction de valeur. Dans un problème de prédiction, il s'agit d'approcher v_π ; dans un problème de contrôle, il s'agit souvent d'approcher q_π ou q_* .

D'un point de vue mathématique, on remplace une grande table par une application

$$(s, a) \mapsto \hat{q}(s, a, \mathbf{w}) \quad \text{ou} \quad s \mapsto \hat{v}(s, \mathbf{w}).$$

L'état s n'est donc plus une simple clé d'accès à une mémoire ; il devient l'entrée d'une fonction.

2.2 Apprentissage avec un oracle

Pour comprendre l'idée, imaginons d'abord une situation idéale. Supposons qu'un oracle nous donne la vraie valeur $v_\pi(s)$ pour tout état s . On pourrait alors définir un critère quadratique (perte ou *loss*), par exemple

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2} \sum_{s \in \mathcal{S}} d(s) (v_\pi(s) - \hat{v}(s, \mathbf{w}))^2,$$

où $d(s)$ est un poids positif indiquant l'importance relative des états.

On retomberait alors sur un problème classique d'approximation : choisir les paramètres $\mathbf{w}$ pour minimiser $\mathcal{L}(\mathbf{w})$. Le vrai problème du RL avec approximation de fonction n'est donc pas de définir une famille de fonctions ; il est de construire des cibles raisonnables alors que l'oracle n'existe pas.

Autrement dit, le RL avec approximation de fonction peut se lire comme :

« régression sur des cibles construites à partir de trajectoires ».

Cette remarque est essentielle, car elle explique pourquoi les méthodes de RL vues dans RL2 restent visibles ici : Monte Carlo, TD, SARSA et Q-learning servent à produire les cibles d'apprentissage.

2.3 Rappel : descente de gradient stochastique

Lorsqu'on dispose d'exemples d'apprentissage, une méthode naturelle pour ajuster des paramètres est la **descente de gradient** (*gradient descent*), puis, en pratique, sa version la plus utilisée : la **descente de gradient stochastique** (*stochastic gradient descent*, SGD).

Considérons un problème général de régression. On observe des couples aléatoires (X, Y) , où X désigne la variable aléatoire d'entrée et Y la variable aléatoire cible associée, et l'on cherche à ajuster une fonction paramétrée $\hat{f}(X, \mathbf{w})$. Un critère classique pour mesurer la qualité des prédictions est l'**erreur quadratique moyenne**, ou **perte quadratique moyenne** (*mean squared error loss*) :

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2} \mathbb{E}[(Y - \hat{f}(X, \mathbf{w}))^2].$$

Le facteur $\frac{1}{2}$ est introduit uniquement pour simplifier les dérivées. Le critère $\mathcal{L}(\mathbf{w})$ est petit lorsque les prédictions $\hat{f}(X, \mathbf{w})$ sont proches des cibles Y , et grand dans le cas contraire.

L'idée de la descente de gradient est de modifier les paramètres dans le sens qui fait décroître ce critère. Le gradient $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$ indique la direction de plus forte augmentation de $\mathcal{L}$. Pour faire diminuer le critère, on déplace donc les paramètres dans la direction opposée :

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t),$$

où $\alpha_t > 0$ est le **pas d'apprentissage** (*learning rate*, ou *step size*). Si α_t est trop grand, on risque de faire des pas excessifs et de rendre l'apprentissage instable ; s'il est trop petit, l'apprentissage devient très lent.

Cette écriture est conceptuellement simple, mais elle pose un problème pratique. Le critère $\mathcal{L}(\mathbf{w})$ est défini comme une moyenne théorique sur la distribution des données, et cette distribution n'est en général pas connue explicitement. On ne peut donc pas calculer directement le gradient exact $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$. En pratique, on le remplace par une approximation construite à partir des données d'apprentissage disponibles.

Si l'on dispose d'un ensemble fini d'exemples

$$\{(x_i, y_i)\}_{i=1}^N,$$

on peut d'abord remplacer le critère théorique par le critère empirique

$$\hat{\mathcal{L}}_N(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{f}(x_i, \mathbf{w}))^2.$$

Calculer son gradient exact revient à parcourir tous les exemples avant chaque mise à jour. Cette approche est possible, mais elle devient coûteuse lorsque N est grand.

C'est pourquoi on utilise le plus souvent une approximation fondée sur des petits sous-ensembles aléatoires de données, appelés **mini-batch**. Si B_t désigne un mini-batch tiré parmi les données d'apprentissage, on considère la perte moyenne sur ce lot :

$$\ell_{B_t}(\mathbf{w}) = \frac{1}{2|B_t|} \sum_{(x,y) \in B_t} (y - \hat{f}(x, \mathbf{w}))^2.$$

La mise à jour devient alors

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla_{\mathbf{w}} \ell_{B_t}(\mathbf{w}_t).$$

C'est précisément ce caractère aléatoire du mini-batch qui justifie le mot *stochastique*. À chaque itération, on ne suit plus exactement le gradient du critère moyen, mais une approximation bruitée de ce gradient. La trajectoire des paramètres devient donc moins régulière, mais chaque mise à jour coûte beaucoup moins cher, ce qui permet d'apprendre efficacement sur de grands ensembles de données. En pratique, on parcourt souvent l'ensemble des données plusieurs fois ; chaque passage complet est appelé une **époque** (*epoch*), et, au cours d'une époque, les exemples sont traités par mini-batches successifs.

Le cas extrême correspond à un mini-batch de taille 1. Si l'on observe au temps t un seul exemple (x_t, y_t) , on définit la **perte quadratique instantanée**

$$\ell_t(\mathbf{w}) = \frac{1}{2} (y_t - \hat{f}(x_t, \mathbf{w}))^2,$$

et la mise à jour devient

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla_{\mathbf{w}} \ell_t(\mathbf{w}_t).$$

En dérivant cette expression, on obtient

$$\nabla_{\mathbf{w}} \ell_t(\mathbf{w}) = -(y_t - \hat{f}(x_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{f}(x_t, \mathbf{w}),$$

d'où

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (y_t - \hat{f}(x_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{f}(x_t, \mathbf{w}_t).$$

Cette formule a exactement la structure familière des méthodes étudiées en RL2 : estimation courante, erreur, pas d'apprentissage, correction. Ce qui change d'un contexte à l'autre, ce n'est pas cette structure générale, mais la manière dont on construit la cible Y_t .

Dans les applications de RL, le cas de mises à jour très locales, parfois fondées sur une seule transition observée, est particulièrement important. Les méthodes de type TD peuvent ainsi être vues comme des méthodes apparentées à la SGD : les données arrivent séquentiellement, on construit une cible à partir de l'expérience courante, puis on corrige immédiatement les paramètres.

Model Free Approximation

Dans le cadre model-free, l'entrée X_t sera un état S_t ou un couple (S_t, A_t) , et la cible Y_t sera construite à partir de la trajectoire observée. On peut ainsi voir l'approximation de fonction model-free comme la combinaison de deux idées :

- une idée de RL, qui fournit des cibles à partir de l'expérience ;
- une idée d'optimisation, qui ajuste les paramètres pour suivre au mieux ces cibles.

Une difficulté supplémentaire apparaît cependant dans les méthodes de type TD ou Q-learning : la cible dépend elle-même, en partie, de l'approximation courante. C'est la raison pour laquelle on parle souvent de **méthodes à semi-gradient** (*semi-gradient methods*) plutôt que de vrai gradient exact.

2.4 Approximation linéaire de la fonction de valeur

L'idée générale de l'approximation de fonction est de remplacer la table de valeurs par une famille de fonctions paramétrées. Plusieurs choix sont possibles pour représenter une fonction de valeur.

On peut par exemple utiliser : les combinaisons linéaires de caractéristiques (*linear function approximation*), qui sont différentiables et mathématiquement simples ; les réseaux de neurones (*neural networks*), eux aussi différentiables, mais beaucoup plus expressifs ; les arbres de décision ; les méthodes des plus proches voisins ; des bases de fonctions, par exemple de Fourier ou d'ondelettes.

Toutes ces approches poursuivent la même idée : au lieu d'apprendre une valeur indépendante pour chaque état, on apprend une fonction paramétrée capable de produire une prédiction à partir d'une représentation de l'état.

Dans ce cours, on se concentrera sur le cas de l'**approximation linéaire**, car c'est le modèle le plus simple permettant de comprendre clairement les principes généraux de l'approximation de fonction.

Un état s n'est pas utilisé directement comme une simple étiquette. On le transforme d'abord en un vecteur de caractéristiques

$$x(s) = (x_1(s), x_2(s), \dots, x_d(s))^T \in \mathbb{R}^d,$$

appelé aussi **descripteur** ou **vecteur de features**.

Les composantes de $x(s)$ décrivent des propriétés numériques jugées utiles pour prédire la valeur. Par exemple : dans un problème de navigation sur grille, certaines caractéristiques pourraient indiquer la distance à la cible, la proximité d'un obstacle ou la présence dans une zone particulière ; dans un problème de contrôle d'un véhicule, elles pourraient contenir la vitesse, l'écart à la trajectoire désirée ou l'orientation du véhicule.

Dans l'approximation linéaire, la valeur prédite est une combinaison linéaire de ces caractéristiques :

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^\top x(s).$$

En développant le produit scalaire, on obtient

$$\hat{v}(s, \mathbf{w}) = \sum_{j=1}^d w_j x_j(s).$$

Cette formule mérite d'être lue lentement. Les quantités $x_j(s)$ décrivent l'état sous forme numérique, tandis que les coefficients w_j mesurent l'influence de chacune de ces caractéristiques sur la valeur prédite. L'état n'est donc plus mémorisé pour lui-même ; il est d'abord transformé en représentation, puis cette représentation est combinée linéairement.

Un avantage essentiel de ce modèle est sa simplicité analytique. Comme la fonction est linéaire par rapport aux paramètres $\mathbf{w}$, son gradient s'écrit immédiatement

$$\nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = x(s).$$

Cette relation jouera un rôle central dans toutes les mises à jour par descente de gradient.

La même idée pourra être utilisée plus tard pour représenter non seulement une valeur d'état $\hat{v}(s, \mathbf{w})$, mais aussi une valeur d'action-état $\hat{q}(s, a, \mathbf{w})$, en remplaçant simplement $x(s)$ par un vecteur de caractéristiques dépendant du couple (s, a) .

2.5 Monte Carlo avec approximation de la fonction de valeur

Les méthodes Monte Carlo sont les plus simples à combiner avec l'approximation de fonction, car leur cible est un retour complet réellement observé. Elles fournissent donc une première manière naturelle d'apprendre les paramètres d'une fonction de valeur approchée.

Sur le plan théorique, si l'on suit une politique fixée π , on peut considérer le critère moyen

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2} \mathbb{E}_{\pi} \left[(G_t - \hat{v}(S_t, \mathbf{w}))^2 \right].$$

Ce critère mesure l'écart quadratique moyen entre la valeur prédite $\hat{v}(S_t, \mathbf{w})$ et le retour G_t . Il faut bien lire cette formule : S_t et G_t y désignent des variables aléatoires produites par les trajectoires générées sous la politique π .

En principe, on pourrait chercher à minimiser directement $\mathcal{L}(\mathbf{w})$. Mais, comme dans le rappel sur la descente de gradient stochastique, cette espérance n'est pas connue explicitement. On l'approche donc à partir des épisodes effectivement observés.

Une fois un épisode réalisé,

$$s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T,$$

on peut calculer pour chaque instant t le retour complet

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{T-t-1} r_T.$$

Bien que l'on soit maintenant sur une trajectoire particulière, on conserve la notation standard G_t pour ce retour calculé à partir des récompenses effectivement observées. On utilise alors G_t comme cible pour ajuster la prédiction $\hat{v}(s_t, \mathbf{w})$.

Autrement dit, la perte instantanée associée à l'instant t est

$$\ell_t(\mathbf{w}) = \frac{1}{2} (G_t - \hat{v}(s_t, \mathbf{w}))^2.$$

La mise à jour pratique correspond alors à une descente de gradient stochastique avec mini-batch de taille 1 :

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla_{\mathbf{w}} \ell_t(\mathbf{w}_t),$$

c'est-à-dire

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (G_t - \hat{v}(s_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w}_t).$$

Dans le cas de l'approximation linéaire,

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^\top x(s), \quad \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = x(s),$$

de sorte que la mise à jour devient

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (G_t - \mathbf{w}_t^\top x(s_t)) x(s_t).$$

Algorithme : Monte Carlo avec approximation linéaire

1. Choisir une représentation $x(s)$ et initialiser $\mathbf{w}$.
2. Générer un épisode complet en suivant la politique fixée π .
3. Pour chaque instant t , calculer le retour complet

$$G_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T.$$

4. Mettre à jour les paramètres par SGD, c'est-à-dire avec un mini-batch de taille 1 :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha (G_t - \mathbf{w}^\top x(s_t)) x(s_t).$$

5. Répéter sur de nouveaux épisodes.

Critères d'arrêt et choix du pas d'apprentissage. Comme pour beaucoup de méthodes itératives, l'algorithme précédent ne dit pas à lui seul quand il faut arrêter les mises à jour. Plusieurs stratégies sont possibles.

La plus simple consiste à fixer à l'avance un budget de calcul, par exemple un nombre maximal d'épisodes ou un nombre maximal de mises à jour. Cette solution est facile à mettre en oeuvre, mais elle ne garantit pas que les paramètres soient effectivement stabilisés au moment de l'arrêt.

Une deuxième possibilité consiste à surveiller l'évolution des paramètres et à arrêter lorsque la variation devient petite, par exemple lorsque

$$\|\mathbf{w}_{k+1} - \mathbf{w}_k\| \leq \varepsilon,$$

pour un seuil $\varepsilon > 0$ choisi à l'avance. Ce critère est utile, mais il doit être interprété avec prudence : une faible variation des paramètres peut aussi provenir d'un pas d'apprentissage très petit, sans que l'on soit encore proche d'une bonne approximation.

On peut également surveiller une quantité empirique liée au critère d'apprentissage, par exemple une moyenne glissante de la perte

$$\frac{1}{2} (G_t - \hat{v}(s_t, \mathbf{w}))^2,$$

ou encore vérifier que les prédictions $\hat{v}(s, \mathbf{w})$ se stabilisent sur un ensemble d'états de référence. Dans un cadre plus orienté contrôle, on peut aussi suivre la performance moyenne obtenue sur des épisodes récents.

En pratique, on combine souvent plusieurs idées : un budget maximal d'entraînement, accompagné d'un test de stabilisation des paramètres, de la perte empirique ou des prédictions.

Le choix du pas d'apprentissage α est lui aussi crucial. Si α est trop grand, les mises à jour risquent d'être trop brutales, ce qui peut produire des oscillations ou de l'instabilité. Si α est trop petit, l'apprentissage devient au contraire très lent. On peut utiliser un pas constant pour conserver des mises à jour simples, ou bien un pas décroissant α_t afin de stabiliser progressivement l'algorithme au fil des itérations. Il existe aussi des méthodes à pas adaptatif, qui ajustent automatiquement l'amplitude des mises à jour, mais cette question dépasse le cadre introductif du présent cours.

Bien entendu, cette discussion dépasse le seul cadre de l'algorithme Monte Carlo présenté ici : elle s'applique plus généralement aux autres méthodes itératives d'apprentissage par approximation de fonction qui seront introduites dans la suite.

Remarque 2.1. Monte Carlo avec approximation de fonction possède un avantage conceptuel important : il n'y a pas de bootstrap, car la cible G_t est un retour réellement observé. La méthode ressemble donc beaucoup à une régression supervisée classique.

Ses limites restent cependant proches de celles du cas tabulaire. Il faut attendre la fin de l'épisode avant de construire la cible, ce qui empêche les mises à jour immédiates. De plus, la variance des retours, déjà discutée dans RL2 [2, section 2.1.5], peut être grande lorsque les épisodes sont longs ou lorsque les récompenses sont bruitées. Ces difficultés motivent le passage aux méthodes de différence temporelle, qui remplacent le retour complet par une cible plus locale disponible après une seule transition.

2.6 TD Learning avec approximation de la fonction de valeur

Les méthodes de différence temporelle se combinent elles aussi naturellement avec l'approximation de fonction. L'idée générale reste la même que dans RL2 [2, section 2.2] : au lieu d'attendre le retour complet, on construit une cible locale inspirée de l'équation de Bellman.

Principe

Pour une politique fixée π , l'équation de Bellman s'écrit [1, section 2.10, p. 15]

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s].$$

L'idée de TD(0) avec approximation est de remplacer la fonction de valeur exacte v_π par une fonction approchée $\hat{v}(\cdot, \mathbf{w})$. On introduit alors la cible TD à un pas

$$Y_t^{\text{TD}} = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t),$$

et l'erreur TD correspondante

$$\delta_t = Y_t^{\text{TD}} - \hat{v}(S_t, \mathbf{w}_t) = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t).$$

Cette écriture doit être lue avec soin. Dans les deux formules précédentes, S_t , S_{t+1} et R_{t+1} désignent encore des variables aléatoires, c'est-à-dire les quantités produites par l'interaction avec l'environnement sous la politique π .

Si l'on observe maintenant une transition particulière

$$s_t, r_{t+1}, s_{t+1},$$

alors la cible TD effectivement utilisée par l'algorithme devient

$$y_t^{\text{TD}} = r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}_t),$$

et l'erreur TD observée vaut

$$\delta_t = r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}_t) - \hat{v}(s_t, \mathbf{w}_t).$$

TD(0) linear value function approximation

Dans le cas de l'approximation linéaire,

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s),$$

la mise à jour naturelle consiste à corriger les paramètres à partir de l'erreur TD observée :

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \delta_t \nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w}_t).$$

Comme

$$\nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = \mathbf{x}(s),$$

on obtient la formule simple

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \delta_t \mathbf{x}(s_t),$$

avec

$$\delta_t = r_{t+1} + \gamma \mathbf{w}_t^\top \mathbf{x}(s_{t+1}) - \mathbf{w}_t^\top \mathbf{x}(s_t).$$

Pourquoi parle-t-on de semi-gradient ? Si l'on essayait d'écrire une perte locale fondée sur la cible TD observée, on obtiendrait formellement

$$\ell_t(\mathbf{w}) = \frac{1}{2} \left(r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w}) \right)^2.$$

Mais cette expression dépend de $\mathbf{w}$ de deux façons :

- par la prédiction courante $\hat{v}(s_t, \mathbf{w})$,
- et aussi par la cible bootstrapée $\hat{v}(s_{t+1}, \mathbf{w})$.

Le *vrai* gradient de cette perte ferait donc intervenir la dérivée des deux termes. Or la méthode TD classique n'utilise pas ce gradient complet. Elle traite la cible

$$r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}_t)$$

comme une constante locale au moment de la mise à jour, et ne dérive que la partie correspondant à la prédiction courante $\hat{v}(s_t, \mathbf{w})$. C'est précisément pour cette raison qu'on parle de **semi-gradient** (*semi-gradient*)¹.

Autrement dit, la mise à jour TD(0) ne suit pas exactement le gradient d'un critère quadratique simple construit à partir de la cible TD ; elle suit une direction plus simple, obtenue en gelant la cible pendant la mise à jour. Cette approximation est exactement ce qui rend le bootstrap facile à utiliser en pratique.

1. Voir aussi l'explication pédagogique [Why "Semi"? Convergence Notes](#) dans la page ApX consacrée aux semi-gradient TD methods.

Convergence : ce qu'il faut retenir. Le fait d'utiliser un semi-gradient signifie qu'on n'est plus, en général, dans le cadre des garanties classiques de la descente de gradient exacte. On ne peut donc pas affirmer, comme dans le cas Monte Carlo, que l'algorithme minimise directement une erreur quadratique moyenne ordinaire.

Cependant, dans le cas important de la **prédiction on-policy avec approximation linéaire**, la situation reste favorable. Les résultats classiques présentés par Sutton et Barto [3, Sec. 9.3] montrent que les méthodes TD linéaires à semi-gradient convergent sous des hypothèses standard, notamment lorsque le pas d'apprentissage décroît de manière appropriée. La convergence n'est pas en général vers le minimiseur exact de l'erreur quadratique moyenne, mais vers une solution bien définie, proche de la meilleure approximation accessible dans la famille linéaire considérée. C'est l'une des raisons majeures pour lesquelles le cas linéaire occupe une place centrale dans la théorie.

Algorithme : TD(0) linéaire pour la prédiction

1. Choisir les caractéristiques $x(s)$ et initialiser $\mathbf{w}$.
2. Exécuter la politique fixée π .
3. À chaque transition observée

$$s_t, r_{t+1}, s_{t+1},$$

calculer l'erreur TD

$$\delta_t = r_{t+1} + \gamma \mathbf{w}^\top x(s_{t+1}) - \mathbf{w}^\top x(s_t).$$

4. Mettre à jour

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta_t x(s_t).$$

5. Continuer ainsi en ligne, transition après transition.

Remarque 2.2. Par rapport à Monte Carlo, le gain essentiel est que l'on n'attend plus la fin de l'épisode : une seule transition suffit pour produire une cible et corriger immédiatement les paramètres. C'est ce qui rend les méthodes TD particulièrement adaptées aux tâches longues ou continues.

En contrepartie, le bootstrap introduit une dépendance de la cible vis-à-vis de la fonction approchée elle-même. C'est précisément cette dépendance qui fait apparaître les méthodes de semi-gradient. Dans le cadre on-policy linéaire, cette difficulté reste maîtrisable ; dans des cadres plus délicats, notamment hors-politique ou non linéaires, elle peut au contraire conduire à des instabilités beaucoup plus sérieuses.

2.7 Contrôle avec approximation de la fonction de valeur

Dans un problème de contrôle, on ne cherche plus seulement à estimer la valeur d'une politique fixée ; on cherche à apprendre à agir de mieux en mieux au cours de l'expérience. Comme dans RL2 [2, chapitre 3], il est alors naturel de travailler avec une approximation de la fonction de valeur d'action,

$$\hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a) \quad \text{ou} \quad \hat{q}(s, a, \mathbf{w}) \approx q_*(s, a).$$

Une politique gloutonne ou ε -gloutonne peut ensuite être obtenue directement à partir de cette approximation, en choisissant dans chaque état une action de valeur prédite élevée.

Contrôle et approximation de la valeur

Le principe général reste celui de la *generalized policy iteration*, déjà introduite dans RL2 [2, section 3.1.1]. Deux mécanismes s'entrelacent :

- une **évaluation approximative**, où l'on ajuste les paramètres $\mathbf{w}$ pour mieux représenter une fonction de valeur ;
- une **amélioration de politique**, où l'on utilise l'approximation courante pour favoriser les actions qui paraissent les plus prometteuses.

Dans le cas tabulaire, ces deux effets modifiaient directement les entrées d'une table. Avec l'approximation de fonction, ils modifient une représentation paramétrée commune à de nombreux couples (s, a) . Une seule mise à jour peut donc affecter simultanément les valeurs prédites dans une grande partie de l'espace état-action.

Malgré cette différence de représentation, le schéma de mise à jour reste fondamentalement le même que dans les sections précédentes. Dans tous les cas, on retrouve une formule du type

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (\text{cible} - \text{prédiction courante}) \nabla_{\mathbf{w}} \hat{q}(\text{entrée}, \mathbf{w}_t).$$

Autrement dit, toute la différence entre les méthodes vient moins de la forme générale de la correction que de la manière dont on construit la cible.

Contrôle avec un oracle

Supposons, fictivement, qu'un oracle fournisse la vraie fonction optimale $q_*(s, a)$. On pourrait alors définir un critère supervisé du type

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2} \sum_{s,a} d(s, a) (q_*(s, a) - \hat{q}(s, a, \mathbf{w}))^2,$$

où $d(s, a)$ est un poids donnant plus ou moins d'importance aux différents couples (s, a) . Une fois les paramètres appris, on construirait une politique gloutonne par

$$\pi_{\mathbf{w}}(s) \in \operatorname{argmax}_{a \in \mathcal{A}} \hat{q}(s, a, \mathbf{w}).$$

Cette expérience de pensée met bien en évidence la vraie difficulté : l'amélioration gloutonne elle-même est simple ; ce qui est difficile est la construction de bonnes cibles d'apprentissage, puisque l'oracle donnant q_* n'existe pas.

SARSA et approximation de la valeur

Dans le cas on-policy, l'analogue direct de SARSA consiste à utiliser comme cible la valeur prédite du couple état-action effectivement rencontré au pas suivant. Sur le plan théorique, on introduit la variable aléatoire

$$Y_t^{\text{SARSA}} = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t),$$

et l'erreur TD correspondante

$$\Delta_t = Y_t^{\text{SARSA}} - \hat{q}(S_t, A_t, \mathbf{w}_t) = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t).$$

Si l'on observe une transition particulière

$$s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1},$$

la cible effectivement utilisée par l'algorithme devient

$$y_t^{\text{SARSA}} = r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t),$$

et l'erreur TD observée vaut

$$\delta_t = r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t) - \hat{q}(s_t, a_t, \mathbf{w}_t).$$

La mise à jour générale à semi-gradient s'écrit alors

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \delta_t \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w}_t).$$

Dans le cas linéaire,

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^\top x(s, a),$$

on obtient la règle simple

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \delta_t x(s_t, a_t).$$

Le sens de cette formule est exactement le même qu'en tabulaire : on apprend la valeur du comportement réellement exécuté, exploration comprise. C'est pourquoi SARSA avec approximation conserve naturellement le caractère prudent déjà observé dans les exemples de RL2 [2, section 3.2].

Rappel sur l'idée de semi-gradient

Comme pour TD(0) en prédiction, les méthodes de contrôle bootstrappées avec approximation n'utilisent pas en général le gradient complet d'une perte quadratique ordinaire. La raison est que la cible elle-même dépend des paramètres $\mathbf{w}_t$.

Par exemple, dans SARSA, la cible

$$r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t)$$

contient déjà la fonction approchée. Si l'on dérivait complètement une perte locale associée à cette cible, il faudrait dériver à la fois la prédiction courante et la cible bootstrapée. La méthode classique procède autrement : au moment de la mise à jour, on *gèle* la cible et l'on ne dérive que la prédiction courante $\hat{q}(s_t, a_t, \mathbf{w})$. C'est précisément cela qu'on appelle un *semi-gradient*.

Cette simplification est très utile, car elle conduit à des règles de mise à jour simples et locales. En revanche, elle signifie aussi que l'algorithme ne suit pas exactement le gradient d'un critère global simple. Dans le cadre linéaire on-policy, comme pour la prédiction TD, cette difficulté reste relativement bien maîtrisée. En revanche, hors-politique, la situation devient beaucoup plus délicate.

Q-learning et approximation de la valeur (cas linéaire)

Dans le cas off-policy, la cible naturelle est celle de Q-learning. Au niveau théorique, on introduit

$$Y_t^Q = R_{t+1} + \gamma \max_{a' \in \mathcal{A}} \hat{q}(S_{t+1}, a', \mathbf{w}_t),$$

puis l'erreur associée

$$\Delta_t = Y_t^Q - \hat{q}(S_t, A_t, \mathbf{w}_t) = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t).$$

Pour une transition observée

$$s_t, a_t, r_{t+1}, s_{t+1},$$

on obtient la cible empirique

$$y_t^Q = r_{t+1} + \gamma \max_{a' \in \mathcal{A}} \hat{q}(s_{t+1}, a', \mathbf{w}_t),$$

et l'erreur TD observée

$$\delta_t = r_{t+1} + \gamma \max_{a'} \hat{q}(s_{t+1}, a', \mathbf{w}_t) - \hat{q}(s_t, a_t, \mathbf{w}_t).$$

La mise à jour à semi-gradient s'écrit alors

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \delta_t \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w}_t).$$

Dans le cas linéaire, cela donne encore

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \delta_t x(s_t, a_t).$$

Formellement, cette formule ressemble beaucoup à celle de SARSA. Toute la différence vient de la cible. SARSA regarde l'action effectivement choisie au temps suivant ; Q-learning regarde l'action qui paraît la meilleure selon l'approximation courante. Le même schéma de correction est donc conservé, mais la façon de construire la cible change profondément la dynamique de l'apprentissage.

Pourquoi la transition vers DQN devient nécessaire

Dans le cas tabulaire, Q-learning est déjà plus délicat que SARSA, notamment à cause du caractère off-policy et de l'opérateur max. Avec approximation de fonction, ces difficultés se renforcent encore.

D'abord, la maximisation

$$\max_{a'} \hat{q}(s_{t+1}, a', \mathbf{w}_t)$$

peut amplifier les erreurs d'estimation : une action peut paraître artificiellement très bonne simplement parce qu'elle a été surestimée par l'approximation courante. Ensuite, la cible utilisée pour mettre à jour les paramètres dépend elle-même de ces mêmes paramètres, ce qui crée un couplage fort entre cible et prédiction. Enfin, les paramètres étant partagés entre de nombreux couples (s, a) , une correction locale peut perturber les prédictions ailleurs dans l'espace d'états et d'actions.

On retrouve ici, sous une forme particulièrement nette, la combinaison déjà signalée plus haut : approximation de fonction, bootstrap, et apprentissage off-policy. C'est précisément cette combinaison qui rend l'apprentissage potentiellement instable.

Deep Q-Network (DQN) peut être vu comme une tentative pour conserver l'idée centrale de Q-learning tout en ajoutant des mécanismes de stabilisation adaptés au cas d'une approximation non linéaire beaucoup plus riche, en particulier un réseau de neurones profond. L'idée générale reste donc la même :

- construire une cible de type Q-learning ;
- comparer cette cible à la prédiction courante ;
- corriger les paramètres dans le sens de cette erreur.

Mais, pour que cette idée devienne praticable avec un modèle très flexible, il faut introduire des artifices supplémentaires, en particulier la *replay memory* et les *target networks*. La section suivante montrera précisément comment ces mécanismes cherchent à limiter les instabilités qui apparaissent lorsque l'on applique trop directement Q-learning avec approximation de fonction.

Chapitre 3

Deep Reinforcement Learning

3.1 Principe de la méthode

Le **deep reinforcement learning** consiste à utiliser des réseaux de neurones profonds pour représenter les fonctions apprises. Dans le cadre de Q-learning, cela revient à remplacer une approximation linéaire

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a)$$

par une approximation neuronale

$$Q(s, a; \theta),$$

où θ désigne l'ensemble des poids et biais du réseau.

L'idée centrale est que le réseau ne sert pas seulement à combiner des caractéristiques ; il peut aussi apprendre lui-même une représentation interne adaptée. Cette capacité est particulièrement intéressante lorsque l'état est riche, par exemple une image ou un empilement de plusieurs images successives.

Mais cette puissance s'accompagne d'une difficulté supplémentaire. Plus la fonction approximante est expressive, plus l'optimisation devient délicate. Les instabilités déjà possibles avec une approximation linéaire peuvent être amplifiées. Le succès de DQN ne vient donc pas seulement du réseau profond ; il vient de la manière dont ce réseau est combiné avec des mécanismes de stabilisation.

3.2 End to end learning of Q

Deep Q-Networks (DQNs)

Un **Deep Q-Network** (DQN) est un réseau qui reçoit l'état en entrée et produit, en sortie, une estimation de la valeur de chacune des actions possibles. Si les actions possibles sont

$$\mathcal{A} = \{a_1, \dots, a_m\},$$

alors, pour un état s , le réseau renvoie le vecteur

$$(Q(s, a_1; \theta), \dots, Q(s, a_m; \theta)).$$

Une politique gloutonne consiste ensuite à choisir l'action de plus grande valeur prédite.

DQNs dans Atari

Le résultat emblématique est celui de Mnih et ses collaborateurs [7]. Le modèle utilisé est un **réseau de neurones convolutionnel** (*convolutional neural network*, CNN), adapté au traitement d'images. L'idée est d'apprendre *de bout en bout* (*end-to-end*) une approximation de la fonction de valeur d'action

$$\hat{q}(s, a, \mathbf{w}),$$

sans construire séparément, à la main, un vecteur de caractéristiques décrivant l'état s . Autrement dit, le réseau reçoit directement les pixels en entrée et apprend lui-même une représentation interne utile pour prédire les valeurs des actions.

L'entrée du réseau est constituée d'un petit empilement d'images successives du jeu, prétraitées et mises à une résolution commune. Cet empilement est important : une image isolée ne suffit pas toujours à reconstruire une information dynamique élémentaire, comme une vitesse ou une direction de mouvement, alors que plusieurs images consécutives permettent déjà de capturer une partie de cette information.

Les premières couches du réseau sont convolutionnelles. Leur rôle est d'extraire progressivement des motifs visuels pertinents : contours, objets, déplacements locaux, structures plus complexes de l'écran de jeu. Ces représentations sont ensuite transmises à des couches entièrement connectées, qui combinent l'information extraite pour produire, en sortie, une estimation des valeurs $\hat{q}(s, a, \mathbf{w})$ pour les différentes actions réalisables. On obtient ainsi, pour un état visuel donné, une valeur prédite pour chaque action possible ; l'agent peut alors choisir l'action de valeur maximale, ou appliquer une stratégie ε -gloutonne pour conserver de l'exploration.

Un point important est que l'architecture générale du réseau et l'essentiel des hyperparamètres sont fixés une fois pour toutes, puis réutilisés sur un grand ensemble de jeux Atari avec relativement peu d'adaptations spécifiques. Cela donne au résultat une portée conceptuelle forte : il ne s'agit pas seulement de résoudre un jeu particulier, mais de montrer qu'un même schéma d'apprentissage peut extraire automatiquement, à partir des pixels, une représentation pertinente de l'état et apprendre les valeurs d'action correspondantes dans des environnements variés.

C'est l'une des raisons pour lesquelles ce travail est considéré comme un acte fondateur du *deep reinforcement learning* moderne.

DQ-learning

Malgré son nom, DQN n'abandonne pas l'idée centrale de Q-learning. Le cœur mathématique reste une cible de Bellman approchée pour la fonction de valeur d'action. La différence est que la fonction Q n'est plus représentée par une table, mais par un réseau de neurones paramétré par un vecteur de poids θ .

À partir d'une transition observée

$$(s_t, a_t, r_{t+1}, s_{t+1}),$$

on peut définir, dans une première version naïve, la cible de type Q-learning

$$y_t^Q = \begin{cases} r_{t+1} & \text{si } s_{t+1} \text{ est terminal,} \\ r_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(s_{t+1}, a'; \theta) & \text{sinon.} \end{cases}$$

On cherche alors à rapprocher la prédiction $Q(s_t, a_t; \theta)$ de cette cible en minimisant une perte quadratique, par exemple

$$\mathcal{L}_t(\theta) = \frac{1}{2} (y_t^Q - Q(s_t, a_t; \theta))^2.$$

En pratique, cette perte est optimisée par descente de gradient stochastique, généralement sur des mini-lots de transitions plutôt que sur une seule transition isolée.

À première vue, cette méthode ressemble donc à une simple régression : on construit une cible, puis on ajuste le réseau pour rapprocher sa prédiction de cette cible. Mais deux difficultés majeures apparaissent immédiatement.

La première est la **corrélacion entre les échantillons**. Les transitions successives issues d'une même trajectoire se ressemblent souvent beaucoup, ce qui viole l'hypothèse implicite d'exemples à peu près indépendants que l'on rencontre souvent en apprentissage supervisé.

La seconde est la **non-stationnarité des cibles**. En effet, la quantité

$$y_t^Q = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta)$$

dépend elle-même des paramètres θ que l'on est en train de modifier. Autrement dit, pendant que l'on ajuste la prédiction, la cible se déplace elle aussi.

Ces deux difficultés peuvent provoquer une forte instabilité, voire une divergence de l'apprentissage. C'est précisément pour limiter ces effets que DQN introduit ensuite deux mécanismes de stabilisation essentiels : l'*experience replay*, qui aide à casser les corrélations entre échantillons, et les *fixed Q-targets*, qui rendent les cibles plus stables pendant un certain temps.

DQNs : Experience replay

Le **rejeu d'expérience** (*experience replay*) consiste à stocker dans une mémoire $\mathcal{D}$ les transitions observées au cours de l'interaction avec l'environnement. Une **transition** est ici un quadruplet de la forme

$$(s_t, a_t, r_{t+1}, s_{t+1}),$$

éventuellement complété par une indication précisant si s_{t+1} est terminal.

Concrètement, la mémoire $\mathcal{D}$ joue le rôle d'un tampon (*replay buffer*) qui conserve un grand nombre de transitions récentes. À mesure que de nouvelles transitions sont observées, elles sont ajoutées au buffer, et les plus anciennes sont supprimées lorsque la capacité maximale est atteinte. L'entraînement du réseau ne se fait alors plus directement sur la dernière transition observée. À la place, on tire aléatoirement dans $\mathcal{D}$ un **mini-lot** de transitions, on construit une cible pour chacune d'elles, on calcule la perte moyenne correspondante, puis on effectue une étape de rétropropagation sur cette perte moyenne.

Au début de l'apprentissage, la mémoire de rejeu $\mathcal{D}$ est vide. En pratique, on commence souvent par la remplir partiellement avant d'effectuer les premières mises à jour du réseau. Ce préremplissage permet de disposer d'un ensemble initial d'expériences plus varié et rend les premiers mini-lots moins corrélés.

Cette idée a deux avantages majeurs.

Premièrement, les transitions successives produites par l'environnement sont fortement corrélées. Deux transitions voisines dans le temps se ressemblent souvent beaucoup, car elles proviennent de la même trajectoire et d'états proches. Or les méthodes de type SGD se comportent en général mieux lorsque les exemples d'entraînement sont moins corrélés. Le tirage aléatoire dans le replay buffer casse donc en grande partie l'ordre temporel des observations et rend l'apprentissage plus régulier.

Deuxièmement, une transition informative peut être réutilisée plusieurs fois. Dans un apprentissage purement en ligne, chaque expérience ne servirait qu'une seule fois, au moment où elle est observée. Avec le rejeu d'expérience, la même transition peut contribuer à plusieurs mises à jour, ce qui améliore l'efficacité en données.

Pour que ce mécanisme soit réellement utile, il est préférable que la mémoire $\mathcal{D}$ soit sensiblement plus grande que la taille des mini-lots utilisés pour l'entraînement. Cela permet de mélanger des transitions provenant de contextes variés. Même lorsque ce rapport n'est pas très grand, le simple fait de rééchantillonner aléatoirement les transitions suffit déjà à réduire fortement les corrélations liées à l'ordre d'observation.

Fixed Q-Targets

Les **cibles Q fixes** (*fixed Q-targets*) constituent le second ingrédient essentiel de DQN. Dans une version naïve de Q-learning approché, les mêmes paramètres interviendraient à la fois dans la prédiction courante $Q(s_t, a_t; \theta)$ et dans la cible

$$r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta).$$

Autrement dit, pendant que l'on essaie d'ajuster la prédiction, la cible elle-même se déplace. Cette non-stationnarité peut engendrer des oscillations, des dérives, voire une divergence de l'apprentissage.

Pour limiter ce phénomène, DQN introduit une copie plus stable du réseau, appelée **réseau cible** (*target network*), paramétrée par θ^- . Les paramètres θ du réseau principal continuent à être mis à jour à chaque étape d'entraînement, mais les cibles sont calculées pendant un certain temps à l'aide de θ^- , maintenu fixe :

$$y_t^{\text{DQN}} = \begin{cases} r_{t+1} & \text{si } s_{t+1} \text{ est terminal,} \\ r_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(s_{t+1}, a'; \theta^-) & \text{sinon.} \end{cases}$$

Ainsi, pendant plusieurs itérations consécutives, les cibles restent inchangées tant que θ^- n'est pas mis à jour. On obtient donc un problème d'ajustement localement plus stable : pendant un certain laps de temps, le réseau principal cherche à se rapprocher de cibles presque fixes. Cela réduit les oscillations et les dérives liées au fait que prédiction et cible évoluent simultanément.

Périodiquement, on remplace alors les paramètres du réseau cible par ceux du réseau principal :

$$\theta^- \leftarrow \theta.$$

Les cibles changent donc par paliers, toutes les N itérations environ, au lieu de changer à chaque mise à jour. Ce fonctionnement en escalier n'est pas un défaut majeur en pratique ; c'est au contraire le prix à payer pour obtenir des cibles plus stables sur des fenêtres successives d'apprentissage.

Le choix de N est un compromis. Si N est trop petit, le réseau cible est recopié trop souvent et l'on se rapproche du cas instable où la cible bouge presque en permanence. Si N est trop grand, les cibles deviennent au contraire trop anciennes et risquent de ralentir l'adaptation du réseau principal. En pratique, N est traité comme un **hyperparamètre** : on le choisit empiriquement, en fonction de l'environnement, de la taille du réseau, du rythme des mises à jour et du niveau de stabilité observé pendant l'entraînement.

DQN Algorithme

On peut résumer DQN sous la forme suivante.

Algorithme : DQN (version conceptuelle)

1. Initialiser les paramètres du réseau en ligne θ .
2. Initialiser le réseau cible avec les mêmes paramètres :

$$\theta^- \leftarrow \theta.$$

3. Initialiser un replay buffer vide $\mathcal{D}$.
4. Répéter :
 - a. observer l'état courant s_t ;
 - b. choisir une action a_t , souvent par une stratégie ε -gloutonne fondée sur $Q(\cdot, \cdot; \theta)$;
 - c. exécuter l'action, observer r_{t+1} et s_{t+1} ;
 - d. stocker la transition $(s_t, a_t, r_{t+1}, s_{t+1})$ dans $\mathcal{D}$;

- e. tirer aléatoirement un mini-lot de transitions dans $\mathcal{D}$;
- f. construire pour chaque transition du mini-lot une cible y_i^{DQN} à l'aide du réseau cible $Q(\cdot, \cdot; \theta^-)$;
- g. mettre à jour θ par descente de gradient stochastique sur la perte moyenne du mini-lot ;
- h. périodiquement, toutes les N itérations, recopier θ dans θ^- .

Il faut bien voir que cette méthode reste entièrement model-free. On n'apprend pas explicitement les probabilités de transition $P(s' | s, a)$ ni une dynamique de l'environnement ; on apprend directement une approximation de la fonction de valeur d'action.

Deux exemples de résultats expérimentaux

Les expériences rapportées par Mnih et ses collaborateurs montrent à la fois l'efficacité globale de DQN sur un grand ensemble de jeux Atari, et l'importance des deux mécanismes de stabilisation introduits plus haut, à savoir le rejeu d'expérience et les cibles fixes. On peut illustrer cela par deux exemples complémentaires.

Exemple 1 : vue d'ensemble sur 49 jeux Atari

La figure 3.1 donne une vue d'ensemble des résultats obtenus sur 49 jeux Atari. Les performances y sont normalisées de façon à fixer le niveau *random* à 0% et le niveau *humain* à 100%. Cette représentation n'utilise donc pas les scores bruts, qui varient fortement d'un jeu à l'autre, mais une échelle relative plus facile à comparer.

Cette figure permet de dégager plusieurs idées importantes.

D'abord, elle montre que DQN n'est pas seulement performant sur un jeu isolé, mais sur une famille très diverse de tâches. C'est un point essentiel : le même schéma algorithmique, avec la même architecture générale et les mêmes grands principes d'entraînement, produit de bonnes performances sur un ensemble étendu de jeux.

Ensuite, on observe que DQN dépasse le meilleur apprenant linéaire sur la quasi-totalité des jeux représentés. Cela suggère que le gain ne vient pas seulement de l'usage de Q-learning, mais aussi de la capacité du réseau profond à construire une représentation plus expressive à partir des pixels.

Enfin, la figure montre que DQN atteint ou dépasse souvent le niveau humain, tout en restant en difficulté sur certains jeux. On voit donc apparaître à la fois la force du modèle et ses limites : il peut découvrir des comportements très performants sur de nombreuses tâches visuelles, mais il reste loin d'être uniformément supérieur à l'humain sur tous les problèmes.

Exemple 2 : contribution des composantes de DQN

Pour comprendre *pourquoi* DQN fonctionne mieux que des variantes plus simples, il est utile d'examiner les résultats d'ablation fournis par Mnih et al [7]. Le tableau 3.1 regroupe dans une même vue les résultats de l'Extended Data Table 3 et de l'Extended Data Table 4 du papier. La colonne *Linéaire* correspond à une approximation linéaire entraînée avec replay et target network ; la colonne *Avec replay, avec cible* correspond au système DQN complet.

Ce tableau met bien en évidence la contribution de chaque composante.

Le point de départ le plus faible est la version *vanilla*, sans replay et sans cible fixe (1ere colonne). Dans cette configuration, les résultats sont systématiquement les plus faibles. Cela confirme que l'apprentissage direct de Q-learning avec un réseau profond est instable et difficile à entraîner.

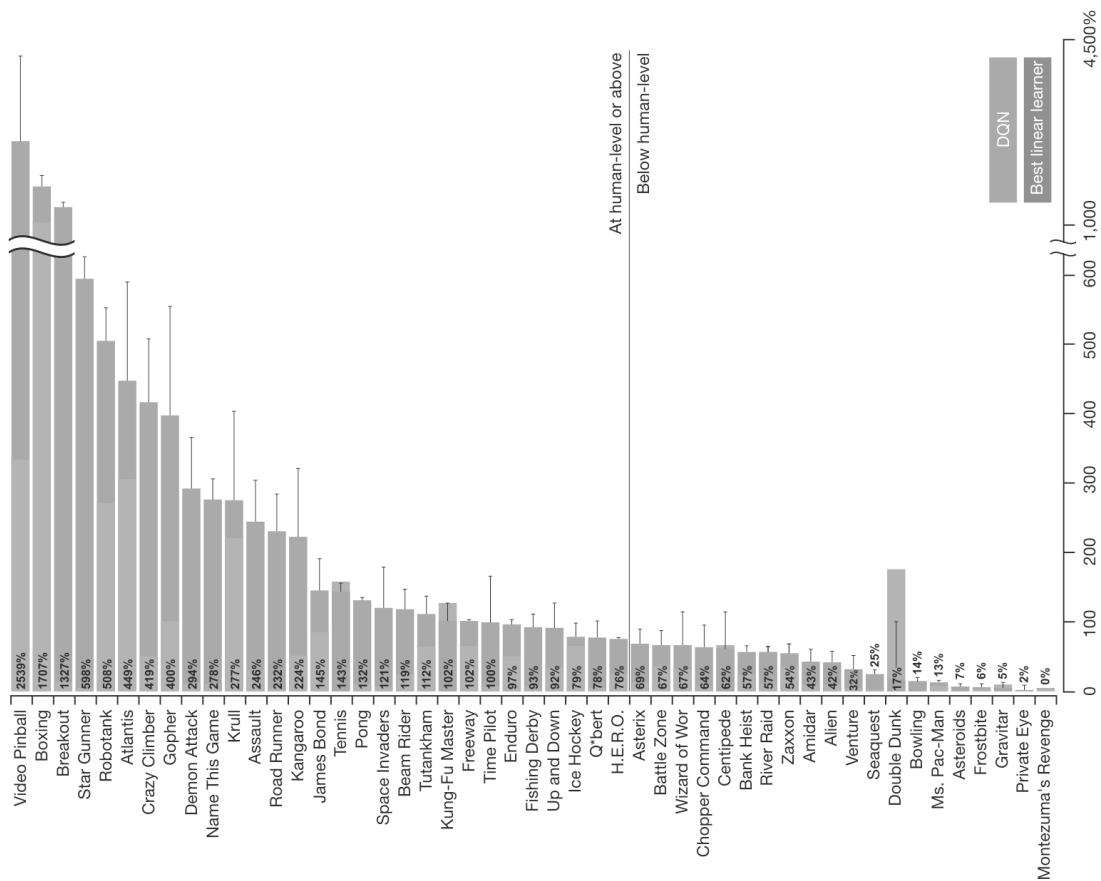


FIGURE 3.1 – Comparaison des performances de DQN avec le meilleur apprenant linéaire sur 49 jeux Atari, d’après la Fig. 3 de [7]. Le niveau 0% correspond à un agent aléatoire, et le niveau 100% à un joueur humain de référence.

L’effet du **rejeu d’expérience** apparaît en comparant les colonnes *sans replay* et *avec replay* (colonnes « 1 vs 3 » et « 2 vs 4 »). Dans les cinq jeux, l’ajout du replay améliore nettement les scores. Le gain est particulièrement spectaculaire sur Breakout, Enduro et Space Invaders. Cela confirme que casser les corrélations temporelles entre transitions et réutiliser les expériences passées rend l’apprentissage beaucoup plus efficace.

L’effet des **cibles fixes** apparaît en comparant, à replay constant, les colonnes *sans cible* et *avec cible* (colonnes « 1 vs 2 » et « 3 vs 4 »). Là encore, l’amélioration est systématique. Elle est parfois modérée, parfois très forte, notamment sur River Raid et Seaquest. On retrouve donc l’idée centrale : stabiliser temporairement les cibles aide le réseau à converger localement au lieu de poursuivre en permanence une cible qui se déplace.

Le meilleur résultat est obtenu quand les deux mécanismes sont réunis : *avec replay, avec cible* (colonne 4). Autrement dit, les deux idées sont utiles séparément, mais leur combinaison est encore meilleure. C’est précisément cette combinaison qui définit le cœur de DQN.

Enfin, la comparaison avec la colonne *Linéaire* montre que ces mécanismes de stabilisation ne suffisent pas à eux seuls : l’architecture profonde apporte elle aussi un gain propre. Même à replay et cibles fixes identiques, DQN domine nettement l’approximation linéaire sur les cinq jeux (colonnes « 4 vs 5 »). L’écart est particulièrement frappant sur Breakout et Enduro, et reste net sur tous les autres jeux du tableau. Cela suggère que le réseau convolutionnel apprend effectivement des représentations visuelles plus utiles que celles fournies par un modèle linéaire.

TABLE 3.1 – Effet du rejeu d’expérience, des cibles fixes et de l’approximation profonde sur cinq jeux de validation, d’après les Extended Data Tables 3 et 4 de [7]. Les valeurs reportées sont des scores moyens d’épisode : pour un jeu donné, plus la valeur est grande, meilleure est la performance, mais les chiffres ne doivent pas être comparés directement entre jeux différents.

Jeu	Baseline (B)	B+Cible Fixe	B+Replay	B+Cible Fixe&Replay	Linéaire
Breakout	3.2	10.2	240.7	316.8	3.0
Enduro	29.1	141.9	831.4	1006.3	62.0
River Raid	1453.0	2867.7	4102.8	7446.6	2346.9
Seaquest	275.8	1003.0	822.6	2894.4	656.9
Space Invaders	302.0	373.2	826.3	1088.9	301.3

Légende. Baseline : sans replay, sans cible fixe ; Linéaire : approximation linéaire avec replay et cible fixe.

3.3 Extensions

Les idées centrales du DQN original de Mnih et al. — approximation profonde de Q , rejeu d’expérience et réseau cible — ont joué un rôle fondateur et se retrouvent dans de nombreuses variantes modernes, comme Double DQN [10], Prioritized Experience Replay [12], Dueling DQN [11], Rainbow [8] ou IQN [9]. Ces méthodes restent des références importantes pour les problèmes à actions discrètes. Dans cette section, nous en présentons synthétiquement quelques-unes.

3.3.1 Double DQN

Q-learning, puis DQN, utilisent dans leur cible un maximum sur des valeurs estimées. Or cette opération peut introduire un **biais de surestimation**. En effet, lorsque plusieurs valeurs sont estimées avec du bruit, le maximum a tendance à sélectionner préférentiellement une estimation affectée par une erreur positive. Autrement dit, parmi plusieurs quantités incertaines, la plus grande n’est pas nécessairement la meilleure au sens réel ; elle est souvent simplement la plus optimiste.

Ce phénomène est déjà présent dans Q-learning tabulaire, mais il devient particulièrement gênant avec l’approximation de fonction, où les erreurs d’estimation peuvent être importantes et corrélées. On obtient alors des cibles trop grandes, ce qui peut dégrader la stabilité de l’apprentissage.

L’idée de **Double Q-learning** est de réduire ce biais en utilisant deux estimations distinctes de Q . L’une sert principalement à *sélectionner* l’action qui paraît la meilleure, l’autre à *évaluer* cette action. Cette dissociation diminue la tendance à réutiliser la même erreur d’estimation à la fois pour choisir et pour évaluer.

Double DQN transpose cette idée au cadre des réseaux de neurones profonds [10]. Au lieu d’utiliser, comme dans DQN standard, le même maximum

$$\max_{a'} Q(s_{t+1}, a'; \theta^-)$$

pour choisir et évaluer l’action suivante, on procède en deux temps :

1. le réseau en ligne, de paramètres θ , choisit l’action qui paraît la meilleure ;
2. le réseau cible, de paramètres θ^- , évalue ensuite cette action.

Pour une transition observée

$$(s_t, a_t, r_{t+1}, s_{t+1}),$$

la cible de Double DQN devient alors

$$y_t^{\text{DDQN}} = \begin{cases} r_{t+1} & \text{si } s_{t+1} \text{ est terminal,} \\ r_{t+1} + \gamma Q(s_{t+1}, \arg\max_{a' \in \mathcal{A}} Q(s_{t+1}, a'; \theta), \theta^-) & \text{sinon.} \end{cases}$$

La différence avec DQN standard est subtile mais importante. Dans DQN, le réseau cible intervient dans un maximum direct, ce qui laisse subsister un effet de surestimation :

$$r_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(s_{t+1}, a'; \theta^-).$$

Dans Double DQN, au contraire, le choix de l'action et son évaluation sont partiellement dissociés. Le réseau en ligne répond à la question : *quelle action semble la meilleure ?* Puis le réseau cible répond à la question : *combien vaut cette action ?*

Cette séparation réduit typiquement le biais d'optimisme et conduit souvent à des estimations plus fiables, donc à un apprentissage plus stable. L'idée générale est proche de celle que l'on rencontre dans d'autres contextes statistiques lorsqu'on combine ou sépare des estimateurs pour limiter un biais introduit par une sélection trop agressive.

Par ailleurs, la dissociation entre le choix d'une action et son évaluation n'est pas sans rappeler, sur le plan conceptuel, l'esprit des méthodes acteur-critique, que nous allons aborder dans le cours suivant : dans les deux cas, on évite de confier tous les rôles à un unique estimateur. Il faut toutefois noter que Double DQN reste une méthode de type Q-learning, sans politique paramétrée séparée ; la séparation y porte seulement sur la sélection et l'évaluation dans la cible de Bellman.

3.3.2 Prioritized Experience Replay

Dans le replay buffer standard, les transitions sont échantillonnées uniformément. Le **rejeu d'expérience priorisé** (*Prioritized Experience Replay*) propose au contraire de revoir plus souvent les transitions qui paraissent les plus instructives [12].

L'idée de départ est simple. Soit i l'indice d'une transition stockée dans le buffer,

$$(s_i, a_i, r_{i+1}, s_{i+1}).$$

Si cette transition produit une grande erreur TD, cela signifie que la prédiction courante du réseau reste encore éloignée de la cible. Une telle transition est donc susceptible d'apporter davantage d'information qu'une transition déjà bien maîtrisée.

On associe alors à chaque transition une **priorité** $p_i > 0$, souvent choisie à partir de la valeur absolue de l'erreur TD :

$$p_i \approx |\delta_i| + \varepsilon,$$

où $\varepsilon > 0$ est une petite constante destinée à éviter qu'une transition ne reçoive une probabilité nulle. Dans le cadre de DQN, on peut par exemple prendre

$$\delta_i = y_i^{\text{DQN}} - Q(s_i, a_i; \theta),$$

avec

$$y_i^{\text{DQN}} = \begin{cases} r_{i+1} & \text{si } s_{i+1} \text{ est terminal,} \\ r_{i+1} + \gamma \max_{a'} Q(s_{i+1}, a'; \theta^-) & \text{sinon.} \end{cases}$$

L'échantillonnage n'est alors plus uniforme. La transition i est tirée avec une probabilité de la forme

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha},$$

où $\alpha \in [0, 1]$ règle l'intensité de la priorisation. Lorsque $\alpha = 0$, on retrouve l'échantillonnage uniforme. Lorsque α augmente, les transitions de grande priorité sont sélectionnées plus souvent.

Concrètement, après chaque mise à jour du réseau, les erreurs TD changent ; les priorités p_i doivent donc être réévaluées régulièrement. Le contenu du buffer n'est donc pas seulement stocké : il est aussi réordonné implicitement par l'importance que l'algorithme attribue à chaque expérience.

Cette stratégie présente un avantage intuitif fort : toutes les transitions n'ont pas la même utilité immédiate pour l'apprentissage. Il paraît donc raisonnable de consacrer davantage d'effort aux expériences qui surprennent encore le modèle.

Comme l'échantillonnage n'est plus uniforme, il introduit toutefois un biais. Pour le compenser partiellement, on utilise en pratique des poids de *importance sampling*, souvent de la forme

$$w_i = \left(\frac{1}{N P(i)} \right)^\beta,$$

où N désigne la taille du buffer et $\beta \in [0, 1]$ règle la force de cette correction. Le détail de cette compensation dépasse le niveau de ce cours ; il suffit ici de retenir l'idée générale : on échantillonne davantage les transitions jugées importantes, puis on corrige partiellement le biais ainsi introduit.

Le rejeu priorisé peut donc être vu comme un raffinement du replay buffer standard : on conserve l'idée de casser les corrélations temporelles par rééchantillonnage, mais on remplace l'échantillonnage uniforme par un échantillonnage orienté vers les expériences les plus informatives.

3.3.3 Dueling DQN

Fonctions valeur et avantage

Dans RL1, on a distingué la valeur d'état $v_\pi(s)$ et la valeur d'action $q_\pi(s, a)$ [1, sections 2.6 et 2.7]. Cette distinction conduit naturellement à introduire la **fonction d'avantage** (*advantage function*) :

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

L'idée intuitive est la suivante. La valeur d'état $v_\pi(s)$ mesure la qualité globale de l'état s . Elle répond à la question : *à quel point la situation présente est-elle favorable ?* En revanche, elle ne dit pas directement quelle action est meilleure qu'une autre dans cet état. Or, dans un problème de contrôle, c'est précisément cette comparaison entre actions qui nous intéresse.

On peut penser à un jeu vidéo. Le simple fait d'être dans une bonne situation — par exemple avec un score élevé, une position favorable ou peu de danger immédiat — indique que l'état s a une grande valeur $v_\pi(s)$. Mais cela ne suffit pas encore à départager deux actions possibles a_1 et a_2 . Il peut arriver que

$$q_\pi(s, a_1) > q_\pi(s, a_2),$$

alors même que la qualité globale de l'état reste inchangée. Autrement dit, la qualité de l'état et l'intérêt relatif des actions sont deux informations liées, mais distinctes.

C'est précisément ce que mesure l'avantage. La quantité $A_\pi(s, a)$ indique ce que le choix de l'action a apporte, en mieux ou en moins bien, par rapport à la seule qualité globale de l'état s . Si $A_\pi(s, a)$ est positif, l'action est meilleure que ce que suggère la seule valeur d'état ; s'il est négatif, elle est moins bonne.

L'identité

$$q_\pi(s, a) = v_\pi(s) + A_\pi(s, a)$$

est élémentaire, mais conceptuellement très utile. Elle sépare :

- ce qui relève de la qualité générale de l'état ;
- ce qui relève du choix relatif entre les actions.

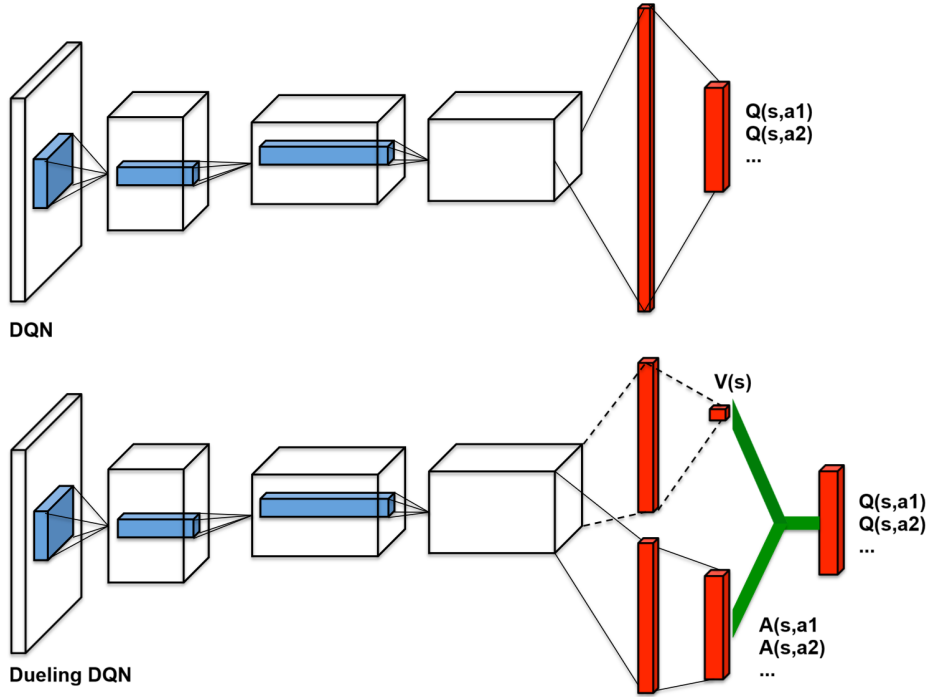


FIGURE 3.2 – Comparaison entre une architecture DQN classique (en haut) et une architecture Dueling DQN (en bas), adaptée de la figure 1 de [11]. Dans la version dueling, le réseau se sépare en deux flux : l'un estime la valeur d'état, l'autre les avantages des actions, avant recombinaison en valeurs Q .

Cette décomposition est particulièrement intéressante dans les situations où de nombreuses actions ont des effets proches. Dans un tel cas, il peut être utile d'apprendre d'abord si l'état est globalement bon ou mauvais, puis seulement de raffiner la différence entre actions. C'est précisément l'idée qui sera exploitée par l'architecture *Dueling DQN*.

Dueling DQN

L'architecture **Dueling DQN** propose de modifier non pas l'algorithme de mise à jour lui-même, mais la manière dont le réseau représente la fonction de valeur d'action [11]. La différence avec un DQN classique est illustrée en figure 3.2. Dans un DQN standard, le réseau produit directement les valeurs

$$Q(s, a_1), Q(s, a_2), \dots$$

pour les différentes actions. Dans un Dueling DQN, après un bloc commun d'extraction de caractéristiques, le réseau se sépare en deux branches :

- une branche estime une valeur d'état $\hat{V}(s)$;
- une branche estime un avantage par action $\hat{A}(s, a)$.

Les deux sorties sont ensuite recombinaisonées par un module d'agrégation pour produire les valeurs d'action finales. Dans cette sous-section, on note $Q(s, a; \theta)$ la sortie du réseau, reconstruite à partir des deux estimateurs internes $\hat{V}(s)$ et $\hat{A}(s, a)$.

L'idée intuitive est la suivante. Dans beaucoup d'états, il est important de savoir si la situation est globalement bonne ou mauvaise, alors que le choix précis entre les actions a peu d'effet. Dans un tel cas, il peut être inefficace de réapprendre séparément toutes les quantités $Q(s, a)$. Il est plus naturel d'apprendre d'abord une estimation de la qualité globale de l'état, puis seulement les écarts relatifs entre actions. C'est précisément ce que permet la décomposition

$$q_\pi(s, a) = v_\pi(s) + A_\pi(s, a).$$

Non-unicité de l'avantage et agrégation valeur-avantage

Une fois les deux branches calculées, il faut recombinaison la valeur d'état et les avantages pour reconstruire les valeurs d'action. La formule la plus naïve serait

$$Q(s, a) = \hat{V}(s) + \hat{A}(s, a).$$

Mais cette écriture pose un problème. Si l'on ajoute une constante C à tous les avantages

$$\hat{A}(s, a_1), \dots, \hat{A}(s, a_n),$$

et que l'on retranche cette même constante à $\hat{V}(s)$, alors toutes les valeurs $Q(s, a)$ restent inchangées. Le réseau peut donc déplacer librement le niveau global entre la branche valeur et la branche avantage, sans que la sortie finale en soit modifiée.

Pour empêcher cela, il faut *ancrer* les avantages par rapport à une référence qui ne soit pas une constante arbitraire, mais une quantité intrinsèque calculée à partir des avantages eux-mêmes. L'idée la plus naturelle consiste à utiliser leur moyenne. On remplace alors chaque avantage par son écart à la moyenne :

$$Q(s, a) = \hat{V}(s) + \left(\hat{A}(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} \hat{A}(s, a') \right).$$

Cette écriture impose automatiquement que la moyenne des avantages corrigés soit nulle. Surtout, elle élimine le degré de liberté additif : si l'on ajoute une constante C à tous les $\hat{A}(s, a)$, alors leur moyenne augmente elle aussi de C , et l'effet s'annule exactement dans la différence

$$\hat{A}(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} \hat{A}(s, a').$$

La branche avantage ne peut donc plus porter un niveau global arbitraire ; elle ne représente plus que les écarts relatifs entre actions. La moyenne des avantages étant soustraite dans le module d'agrégation, cette composante moyenne est absorbée par la branche valeur. En particulier, avec cette normalisation, $\hat{V}(s)$ coïncide avec la moyenne des quantités $Q(s, a)$ sur les actions, tandis que $\hat{A}(s, a)$ mesure seulement l'écart de l'action a à ce niveau moyen.

Une autre possibilité, discutée dans le papier de Wang et al., consiste à soustraire non pas la moyenne des avantages, mais leur maximum :

$$Q(s, a) = \hat{V}(s) + \left(\hat{A}(s, a) - \max_{a' \in \mathcal{A}} \hat{A}(s, a') \right).$$

Cette formule force la meilleure action à avoir un avantage nul. Elle garde une interprétation plus directe, mais la version centrée par la moyenne est en pratique plus stable. C'est pourquoi Wang et al. utilisent cette dernière dans leurs expériences [11].

Il faut enfin souligner que cette agrégation fait partie intégrante du réseau : ce n'est pas une étape algorithmique séparée. Le Dueling DQN garde donc la même interface d'entrée-sortie qu'un Q-réseau classique. On peut l'entraîner avec les mêmes méthodes qu'auparavant, par exemple DQN ou DQN avec replay priorisé, mais avec une représentation interne mieux adaptée aux situations où de nombreuses actions ont des valeurs proches.

Exemple : effet de l'architecture Dueling sur Atari

La figure 3.3 illustre l'effet pratique de l'architecture *dueling* sur un grand ensemble de jeux Atari. Elle représente, pour chaque jeu, l'amélioration obtenue par le réseau dueling par rapport à une architecture classique à flux unique. Les barres orientées vers la droite indiquent un gain en performance.

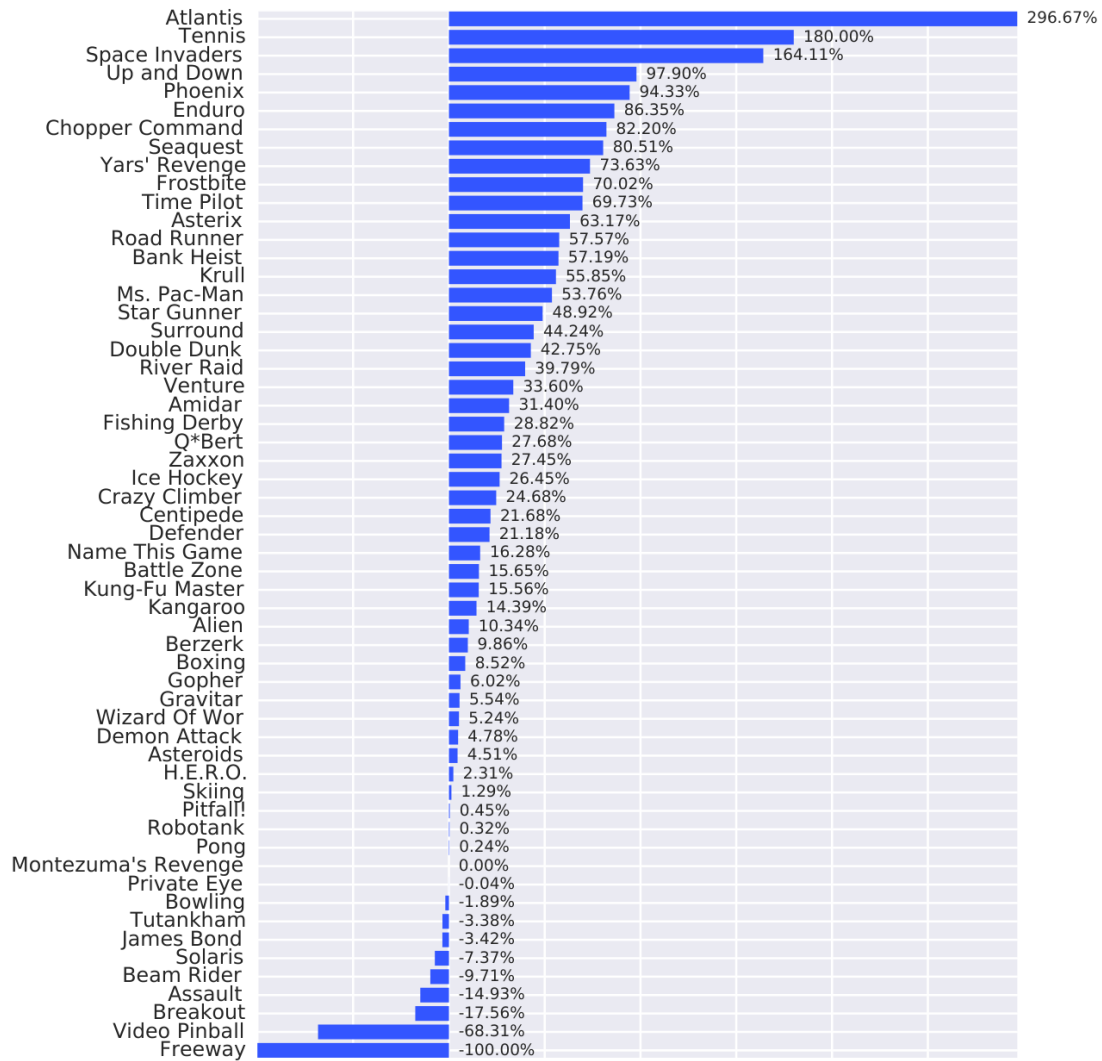


FIGURE 3.3 – Amélioration de l’architecture dueling par rapport à la baseline à flux unique, adaptée de la figure 4 de [11].

Cette figure met en évidence plusieurs points importants. D’abord, le gain est positif sur une large majorité de jeux : 43 des 57 jeux dans une comparaison à capacité similaire (soit 75.4%). L’architecture *dueling* n’est donc pas un raffinement anecdotique, mais une amélioration réellement utile en pratique. Ensuite, l’ampleur du bénéfice dépend fortement du jeu considéré. Sur certains problèmes, l’amélioration est modérée ; sur d’autres, elle est très marquée.

Cette variabilité est cohérente avec l’intuition théorique de la méthode. Le dueling network est particulièrement avantageux lorsque, dans de nombreux états, il est important d’estimer correctement la qualité globale de la situation, alors que les différences entre actions sont faibles ou n’importent qu’à certains moments. Dans ce type de contexte, séparer une branche valeur et une branche avantage permet d’apprendre plus efficacement qu’un réseau qui doit estimer directement toutes les quantités $Q(s, a)$ sans cette structure interne.

La figure ne doit toutefois pas être lue comme une garantie d’amélioration uniforme sur tous les jeux. Elle montre plutôt que l’architecture dueling constitue un biais de modélisation souvent favorable, en particulier lorsque beaucoup d’actions ont des effets proches dans de nombreux états. C’est ce qui explique qu’elle soit devenue une extension classique des architectures de type DQN.

Transition vers Policy Gradient et Actor-Critic

Jusqu'ici, depuis RL1 et RL2, l'objet central était une fonction de valeur à partir de laquelle on dérivait une politique. Les méthodes de **gradient de politique** (*policy gradient*) changent de point de vue : elles paramètrent directement la politique,

$$\pi(a | s; \theta),$$

et ajustent les paramètres de manière à améliorer la récompense espérée.

Les méthodes **actor-critic** combinent ensuite les deux idées. Un *acteur* (*actor*) représente la politique, tandis qu'un *critique* (*critic*) estime une quantité de valeur utilisée pour guider l'apprentissage de l'acteur. Dans ce cadre, la notion d'avantage joue souvent un rôle central, car elle permet d'évaluer si une action choisie est meilleure ou moins bonne que ce que suggère la seule qualité globale de l'état.

De ce point de vue, les développements précédents préparent déjà naturellement la suite. Avec Dueling DQN, on a vu apparaître une séparation entre niveau global de l'état et écarts relatifs entre actions ; avec les méthodes actor-critic, on séparera cette fois la politique elle-même et le mécanisme d'évaluation qui la guide. Il ne s'agit pas de la même architecture ni de la même famille de méthodes, mais il y a bien une continuité conceptuelle. On peut donc voir Dueling DQN comme une transition naturelle entre l'approche purement orientée valeur et les méthodes actor-critic.

Conclusion

Le passage du tabulaire à l'approximation de fonction ne modifie pas l'objectif fondamental du RL. On cherche toujours à estimer des valeurs et à en déduire de meilleures décisions. Ce qui change, c'est la manière de représenter ces valeurs : au lieu d'une table exhaustive, on apprend un nombre plus réduit de paramètres partagés capables de généraliser entre états.

Cette idée apparaît déjà dans les approximations linéaires, où des caractéristiques choisies à la main résument l'information jugée utile. Elle devient beaucoup plus puissante avec les réseaux de neurones profonds, capables d'apprendre eux-mêmes des représentations internes adaptées à la tâche. Mais cette puissance s'accompagne de difficultés nouvelles : instabilités, surestimations, corrélations fortes dans les données, interaction délicate entre approximation et bootstrap.

DQN, Double DQN, Prioritized Experience Replay et Dueling DQN peuvent être vus comme des réponses progressives à ces difficultés. Ils prolongent, à grande échelle, les mêmes idées que celles étudiées dans RL1 et RL2 : cibles de Bellman approchées, erreurs TD, apprentissage guidé par l'expérience, et compromis entre exploration et exploitation.

La suite naturelle du cours est alors de se demander non plus seulement comment apprendre une fonction de valeur puissante, mais comment apprendre directement une politique paramétrée, puis comment coupler politique et critique dans les méthodes actor-critic.

Bibliographie

- [1] Marin Ferecatu, *Apprentissage par renforcement : cadre général et programmation dynamique*, notes de cours RL1, UE RCP211, LeCNAM, Paris, 2026.
- [2] Marin Ferecatu, *Apprentissage par renforcement : prédiction et contrôle Model Free*, notes de cours RL2, UE RCP211, LeCNAM, Paris, 2026.
- [3] Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning : An Introduction*, 2nd edition, The MIT Press, 2018.
<http://incompleteideas.net/book/the-book-2nd.html> (Licence : Creative Commons Attribution-NonCommercial-NoDerivs 2.0 Generic.)
- [4] Shiyu Zhao, *Mathematical Foundations of Reinforcement Learning*, Springer, 2026.
Projet associé : <https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning>
- [5] David Silver, *Lectures on Reinforcement Learning*, 2015.
<https://www.davidsilver.uk/teaching/>
- [6] Christopher J. C. H. Watkins and Peter Dayan, “Q-learning”, *Machine Learning*, 8(3–4), 1992, p. 279–292.
- [7] Volodymyr Mnih et al., “Human-level control through deep reinforcement learning”, *Nature*, 518, 2015, p. 529–533.
- [8] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar et D. Silver, “Rainbow : Combining Improvements in Deep Reinforcement Learning”, dans *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [9] W. Dabney, G. Ostrovski, D. Silver et R. Munos, “Implicit Quantile Networks for Distributional Reinforcement Learning”, dans *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [10] Hado van Hasselt, Arthur Guez and David Silver, “Deep Reinforcement Learning with Double Q-learning”, *Proceedings of AAAI*, 2016.
- [11] Ziyu Wang et al., “Dueling Network Architectures for Deep Reinforcement Learning”, *Proceedings of ICML*, 2016.
- [12] Tom Schaul, John Quan, Ioannis Antonoglou and David Silver, “Prioritized Experience Replay”, *Proceedings of ICLR*, 2016.